



Birla Institute of Technology & Science, Pilani
Dubai Campus

OBJECT-ORIENTED PROGRAMMING ASSIGNMENT
(CS F213)

Semester 1 2025- 2026

REPORT ON
Cricket Attax Elite: Cricket Trump Card Game



SUBMITTED BY: TEAM VOID

2024A7PS0291U - Syed Mohammed Rehan

2024A7PS0300U - Akhil Suravarapu

2024A7PS0314U - Mohammed Zayed Khan

2024A7PS0281U - Subbramaniam Ramesh Shunmugam

2024A7PS0173U – Tejaswi Raghavendra Banda

SUBMITTED TO:

- Prof. Sujala D Shetty
- Prof. Sapna Sadhwani

DATE OF SUBMISSION: 30th November 2025

TABLE OF CONTENTS

- 1. Abstract**
- 2. Introduction**
- 3. System Analysis and Design**
- 4. Implementation**
- 5. Conclusion and Future work**
- 6. References**

1. ABSTRACT

This project is aimed at creating a strategy-centric cricket combat game, which was an opportunity to apply the principles of Object-Oriented programming (OOP) using Java as the core programming language. Through the whole project, we explored how Java can make theoretical concepts into practical application development, building highly performant and intuitive software applications by utilizing Java's comprehensive architecture.

Java was chosen due to its adaptability across platform compatibility and strong support for the principles of OOP. Tools such as the Java Swing, offers a reliable, independent foundation for the GUI(graphical user interface), it ensures the user a smooth and easy to navigate with the buttons, text on field, menus, windows and many more, GUI's can be created using built in libraries and the java swing or the javaFx which provides the tools and components to design the interface.

2. INTRODUCTION

1. Background & Motivation

Trump cards, in general, are a popular social pastime and engaging game for many. It includes a deck of cards, outranking another by comparing the stats on the card. Integrating cricket into this format can make it much more interesting as players, who are enthusiasts, get to compare the real-life stats of their favorite players, making a stronger connection to the game. They evoke a sense of nostalgia and offer a strategic element. Existing offline versions tend to have outdated stats and become monotonous over a period of time.

This project aims to create an interactive cricket card game, "Cricket Attax Elite," using Java concepts with an AI opponent and an interactive GUI.

2. Objectives

- Develop a full-fledged Java OOP application that has a GUI.
- Apply design patterns such as Factory, Strategy, Observer, and State.
- Reproduce the simulated logic of real cricket player matches with roles and stat modifiers.
- Design an AI that plays with a statistical advantage. deliver continuity , good scores, and history.
- Provide a user experience that is interactive and with real-time feedback.

2. Scope

- Single-player mode: Human vs Computer.
- 10 cards per deck for each player.
- Automation of turn flow, card drawing, stat selection, and winner comparison.
- No networking or multiplayer over the internet.
- Desktop-based Java Swing application.

3. SYSTEM ANALYSIS AND DESIGN

1. Requirement Analysis

- **Functional Requirements**

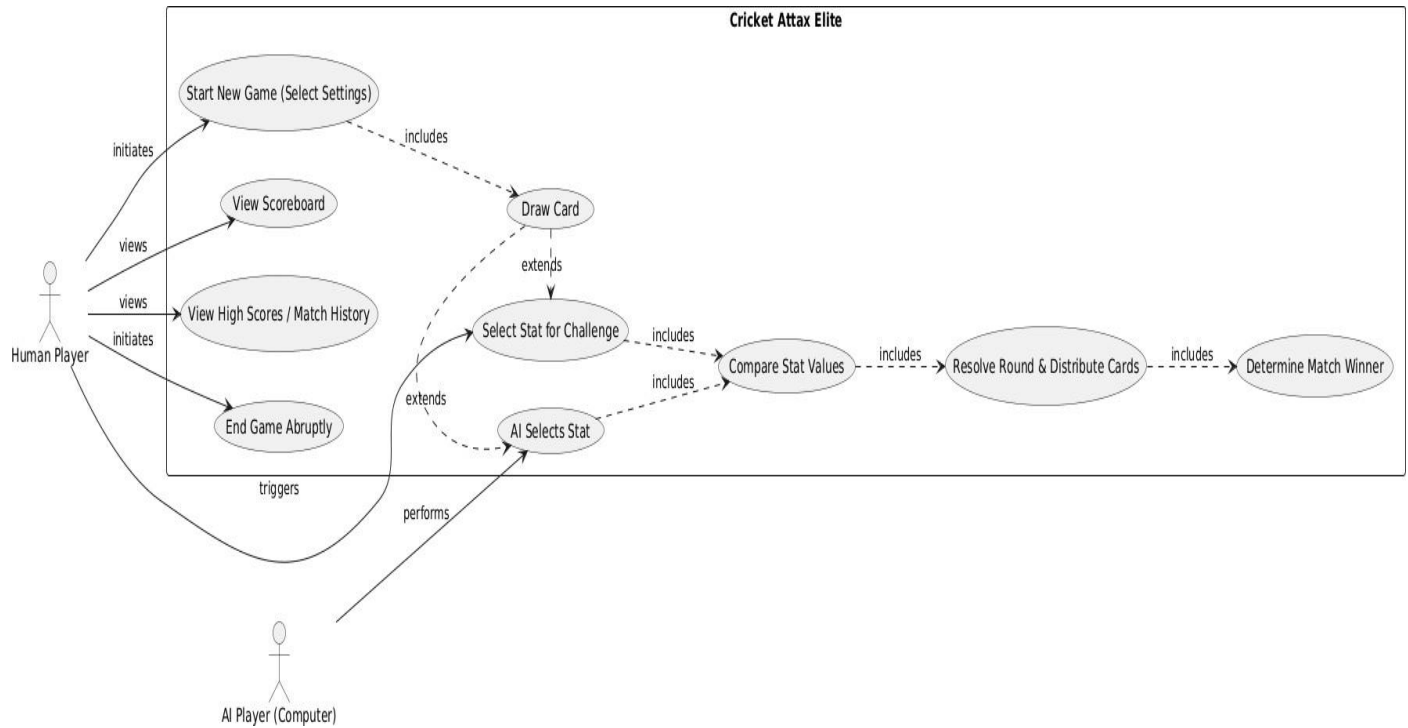
- Assign cards of a player with his or her role and statistics.
- The gamer has the right to play or quit the game.
- The selection of the stats is made by AI alone.
- Compare the stats in every round and give the winner.
- Keep decks, piles of prizes, and turn of events.
- Visual elements: cards, statistics, history and score.
- Save and load
- High scores
- Last match history
- Enabling the user to consider the history of the previous matches

- **Non-Functional Requirements**

- **User friendliness:** Basic interface and procedural directions.
- **Reliability:** Must safely deal with invalid data.
- **Performance:** No lag during play on the GUI.
- **Maintainability:** Listing code with well defined classes.
- **Persistence:** Store data by means of serialization.

3. SYSTEM ANALYSIS AND DESIGN

3.2 Use Case Diagram



4. IMPLEMENTATION

4.1 JAVA concepts used

Inheritance	Player Cards extend AbstractPlayerCard
Interfaces	Card, AIStrategy, GameObserver, to define contracts for component behaviors
Polymorphism	Strategy Pattern for AI, State Pattern for turn flow
Encapsulation	Private attributes with getters/setters
Exception Handling	File I/O (high scores, match history)
File Handling	Serialization for highscores and history
Collections API	Lists, Maps, Queues for decks and stats
Generics	Factory and deck operations
Swing GUI	JFrame, JButton, JTextArea, JPanel
Threads/Timers	AI-thinking delay using Swing Timer
Singleton Pattern	GameEngine ensures one controller instance exists

4. IMPLEMENTATION

4.2 Major Modules

- **4.2.1 Card & Player System:**
 - Enum: PlayerType, StatCategory
 - Abstract class: AbstractPlayerCard
 - Concrete class: ElitePlayerCard
 - Purpose: Model cards of players in the cricket game with real statistics.
- **4.2.2 PlayerCardFactory:**
 - Reads embedded CSV data.
 - Converts rows into ElitePlayerCard objects.
 - Creates 60-card master deck.
- **4.2.3 DeckManager:**
 - Shuffles and deals cards.
 - Returns 11 cards each for Human & AI.
- **4.2.4 AI System (Strategy Pattern):**
 - RandomStrategy: easy mode
 - MaxAdvantageStrategy: hard mode
 - AI makes statistically strongest choice.
- **4.2.5 Game Flow (State Pattern)**
 - 4 States:
 - SelectionState
 - ComparisonState
 - ResolutionState
 - GameOverState
 - Controls interactions for each phase.
- **4.2.6 Observer Pattern**
 - GUI registers as observer → auto updates:
 - scores
 - turns
 - card displays
 - status messages
 - history

4.2.7 GUI

- Built using Swing:
 - Card panels showing stats
 - Buttons for stat selection
 - History window
 - High score & match history popups

5. Conclusion and Future work

1. Conclusion:

The Cricket Attax Elite project considered by us gave us a hands-on and intensive masterclass on Object-Oriented Design and its Design patterns. The field experience regarding the ways of applying the various strategies, tactics, observer patterns to establish good scalable application that is maintainable. The project, in particular, taught us to use the polymorphism and the abstraction to decouple the system parts and turn the theoretical OOP knowledge into the actual application of the code.

2. Future Improvements:

- Adding a real-time PvP feature where players get to compete with one another.
- Introducing enhanced animations and UI styling.
- Introducing collectible cards that players get to unlock gradually by winning games
- Designing limited-time game modes that are different themed, along with online tournaments and leagues
- Add difficulty levels beyond statistical advantage and improving players' stats.

SOURCE CODE

```
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.awt.image.BufferedImage;
import java.io.*;
import java.util.*;
import java.util.List;
import java.util.concurrent.ThreadLocalRandom;
import javax.swing.*;
import javax.swing.border.TitledBorder;

public class CricketAttaxElite {

    public enum GameMode implements Serializable {
        AIVSCOMPUTER("AI vs. Computer (1 Player)");
        private final String displayName;

        GameMode(String displayName) {
            this.displayName = displayName;
        }

        @Override
        public String toString() {
            return displayName;
        }
    }

    public enum StatCategory implements Serializable {
        BATTINGAVG("Batting Avg", true),
        STRIKERATE("Strike Rate", true),
        BOWLINGWICKETS("Wickets Taken", true),
        ECONOMY("Economy Rate", false),
        FIELDINGCATCHES("Fielding Catches", true),
        EXPERIENCE("Experience Yrs", true);

        private final String displayName;
        private final boolean higherIsBetter;
    }
}
```

```

StatCategory(String displayName, boolean higherIsBetter) {
    this.displayName = displayName;
    this.higherIsBetter = higherIsBetter;
}

public String getDisplayName() {
    return displayName;
}

public boolean isHigherIsBetter() {
    return higherIsBetter;
}
}

public enum PlayerType implements Serializable {
    BOWLER, BATSMAN, ALLROUNDER, WICKETKEEPER
}

public interface Card extends Serializable {
    String getPlayerName();
    PlayerType getPlayerType();
    double getStat(StatCategory category);
    Map<StatCategory, Double> getAllStats();
}

public static abstract class AbstractPlayerCard implements Card {
    protected String playerName;
    protected PlayerType type;
    protected Map<StatCategory, Double> stats;

    public AbstractPlayerCard(String playerName, PlayerType type, Map<StatCategory,
Double> stats) {
        this.playerName = playerName;
        this.type = type;
        this.stats = Collections.unmodifiableMap(stats);
    }

    @Override
    public String getPlayerName() {
        return playerName;
    }
}

```

```

    }

    @Override
    public PlayerType getPlayerType() {
        return type;
    }

    @Override
    public double getStat(StatCategory category) {
        return stats.getOrDefault(category, 0.0);
    }

    @Override
    public Map<StatCategory, Double> getAllStats() {
        return stats;
    }
}

public static class ElitePlayerCard extends AbstractPlayerCard {
    public ElitePlayerCard(String playerName, PlayerType type, Map<StatCategory, Double>
stats) {
        super(playerName, type, stats);
    }
}

public static class PlayerCardFactory {
    private static final String FILEPATH = "playerstats.csv";

    private static final String DEFAULTPLAYERDATACSV =
        "Player,Type,BATTING,STRIKE,BOWLING,ECONOMY,FIELDING,EXPERIENCE\n" +
        "Virat Kohli,BATSMAN,55.0,135.5,0.5,8.0,150.0,15.0\n" +
        "Rohit Sharma,BATSMAN,48.0,140.0,1.0,8.5,120.0,14.0\n" +
        "Jasprit Bumrah,BOWLER,5.0,90.0,250.0,6.5,50.0,10.0\n" +
        "Hardik Pandya,ALLROUNDER,35.0,145.0,100.0,7.5,80.0,9.0\n" +
        "Ravindra Jadeja,ALLROUNDER,30.0,125.0,150.0,7.0,200.0,12.0\n" +
        "MS Dhoni,WICKETKEEPER,40.0,130.0,0.0,0.0,300.0,18.0\n" +
        "Suryakumar Yadav,BATSMAN,42.0,155.0,0.0,0.0,70.0,6.0\n" +
        "Mohammed Shami,BOWLER,4.0,75.0,180.0,6.8,40.0,11.0\n" +
        "KL Rahul,WICKETKEEPER,45.0,138.0,0.0,0.0,150.0,10.0\n" +
        "Trent Boult,BOWLER,8.0,100.0,220.0,6.2,30.0,13.0\n" +

```

"Kane Williamson,BATSMAN,50.0,115.0,0.0,0.0,90.0,14.0\n" +
"Ben Stokes,ALLROUNDER,40.0,138.0,120.0,7.2,100.0,11.0\n" +
"Steve Smith,BATSMAN,47.0,120.0,1.0,8.0,110.0,13.0\n" +
"Pat Cummins,BOWLER,20.0,110.0,190.0,7.0,60.0,10.0\n" +
"Quinton de Kock,WICKETKEEPER,38.0,142.0,0.0,0.0,280.0,10.0\n" +
"Rashid Khan,ALLROUNDER,15.0,140.0,200.0,6.0,70.0,8.0\n" +
"Babar Azam,BATSMAN,52.0,125.0,0.0,0.0,80.0,9.0\n" +
"Shaheen Afridi,BOWLER,10.0,80.0,230.0,6.7,45.0,7.0\n" +
"Jos Buttler,WICKETKEEPER,46.0,148.0,0.0,0.0,250.0,10.0\n" +
"David Warner,BATSMAN,44.0,133.0,0.0,0.0,130.0,15.0\n" +
"Jonny Bairstow,BATSMAN,41.0,145.0,0.0,0.0,100.0,9.0\n" +
"Shubman Gill,BATSMAN,51.0,130.0,0.0,0.0,85.0,5.0\n" +
"Rishabh Pant,WICKETKEEPER,36.0,150.0,0.0,0.0,220.0,7.0\n" +
"Yuzvendra Chahal,BOWLER,3.0,60.0,170.0,7.2,35.0,8.0\n" +
"Moeen Ali,ALLROUNDER,32.0,130.0,125.0,7.8,95.0,13.0\n" +
"Sam Curran,ALLROUNDER,28.0,140.0,110.0,8.0,75.0,6.0\n" +
"Glenn Maxwell,ALLROUNDER,38.0,155.0,80.0,8.5,105.0,11.0\n" +
"Faf du Plessis,BATSMAN,43.0,128.0,0.0,0.0,115.0,14.0\n" +
"Adam Zampa,BOWLER,2.0,50.0,165.0,6.9,25.0,7.0\n" +
"Kagiso Rabada,BOWLER,9.0,85.0,210.0,6.6,55.0,9.0\n" +
"Lokesh Rahul,WICKETKEEPER,45.0,138.0,0.0,0.0,150.0,10.0\n" +
"David Miller,BATSMAN,37.0,148.0,0.0,0.0,90.0,12.0\n" +
"Chris Woakes,ALLROUNDER,25.0,120.0,130.0,7.4,85.0,10.0\n" +
"Shaun Marsh,BATSMAN,39.0,122.0,0.0,0.0,100.0,15.0\n" +
"Tom Latham,WICKETKEEPER,35.0,110.0,0.0,0.0,200.0,11.0\n" +
"Shakib Al Hasan,ALLROUNDER,33.0,128.0,140.0,6.3,180.0,16.0\n" +
"Mitchell Starc,BOWLER,12.0,105.0,240.0,6.1,40.0,12.0\n" +
"Tim Southee,BOWLER,15.0,95.0,200.0,6.4,50.0,14.0\n" +
"Shimron Hetmyer,BATSMAN,34.0,160.0,0.0,0.0,60.0,5.0\n" +
"Nicholas Pooran,WICKETKEEPER,30.0,155.0,0.0,0.0,190.0,7.0\n" +
"Kieron Pollard,ALLROUNDER,31.0,150.0,90.0,8.2,110.0,17.0\n" +
"Aaron Finch,BATSMAN,40.0,135.0,0.0,0.0,95.0,15.0\n" +
"Eoin Morgan,BATSMAN,35.0,140.0,0.0,0.0,80.0,16.0\n" +
"Dinesh Karthik,WICKETKEEPER,39.0,132.0,0.0,0.0,270.0,18.0\n" +
"Bhuvneshwar Kumar,BOWLER,10.0,90.0,180.0,7.0,65.0,13.0\n" +
"Ishan Kishan,BATSMAN,40.0,148.0,0.0,0.0,70.0,6.0\n" +
"Sanju Samson,WICKETKEEPER,43.0,140.0,0.0,0.0,210.0,9.0\n" +
"Ravi Ashwin,ALLROUNDER,20.0,110.0,160.0,7.1,80.0,15.0\n" +
"Axar Patel,ALLROUNDER,22.0,130.0,140.0,7.5,75.0,8.0\n" +
"Deepak Chahar,BOWLER,15.0,115.0,155.0,7.6,60.0,7.0\n" +
"T Natarajan,BOWLER,5.0,70.0,190.0,7.3,45.0,5.0\n" +

```

        "Mohammad Rizwan,WICKETKEEPER,47.0,130.0,0.0,0.0,290.0,9.0\n" +
        "Fakhar Zaman,BATSMAN,38.0,142.0,0.0,0.0,110.0,7.0\n" +
        "Shadab Khan,ALLROUNDER,25.0,135.0,130.0,6.8,90.0,7.0\n" +
        "Haris Rauf,BOWLER,7.0,90.0,205.0,6.5,50.0,5.0";

```

```

public List<ElitePlayerCard> loadAll() {
    List<ElitePlayerCard> allCards = new ArrayList<>();
    String dataToProcess = DEFAULTPLAYERDATACSV;
    String[] lines = dataToProcess.split("\n");
    if (lines.length < 2) return allCards;

    try {
        String[] headers = lines[0].split(",");
        Map<String, Integer> headerMap = new HashMap<>();
        for (int i = 0; i < headers.length; i++) {
            headerMap.put(headers[i].trim(), i);
        }

        for (int j = 1; j < lines.length; j++) {
            String line = lines[j];
            if (line.trim().isEmpty()) continue;

            String[] values = line.split(",");
            String playerName = values[headerMap.get("Player")].trim();
            PlayerType type =
                PlayerType.valueOf(values[headerMap.get("Type")].trim().toUpperCase().replace(" ", "_"));
            Map<StatCategory, Double> stats = new HashMap<>();
            stats.put(StatCategory.BATTINGAVG, getStatValue(values, headerMap,
                "BATTING"));
            stats.put(StatCategory.STRIKERATE, getStatValue(values, headerMap, "STRIKE"));
            stats.put(StatCategory.BOWLINGWICKETS, getStatValue(values, headerMap,
                "BOWLING"));
            stats.put(StatCategory.ECONOMY, getStatValue(values, headerMap, "ECONOMY"));
            stats.put(StatCategory.FIELDINGCATCHES, getStatValue(values, headerMap,
                "FIELDING"));
            stats.put(StatCategory.EXPERIENCE, getStatValue(values, headerMap,
                "EXPERIENCE"));

            allCards.add(new ElitePlayerCard(playerName, type, stats));
        }
    } catch (Exception e) {

```

```

        System.err.println("FATAL: Data parsing failed. Error: " + e.getMessage());
    }

    if (allCards.size() < 40) {
        System.err.println("WARNING: Insufficient players loaded (got " + allCards.size() + ").
Game may be unstable.");
    }
    return allCards;
}

private double getStatValue(String[] values, Map<String, Integer> headerMap, String key) {
    if (headerMap.containsKey(key)) {
        try {
            return Double.parseDouble(values[headerMap.get(key)].trim());
        } catch (Exception e) {
            return 0.0;
        }
    }
    return 0.0;
}
}

public static class DeckManager {
    private List<ElitePlayerCard> masterDeck;

    public DeckManager() {
        PlayerCardFactory factory = new PlayerCardFactory();
        this.masterDeck = factory.loadAll();
    }

    public List<List<Card>> dealCards(int cardsPerPlayer) {
        if (masterDeck.size() < cardsPerPlayer * 2) {
            throw new IllegalStateException("Not enough unique players to deal " +
cardsPerPlayer +
            " cards to two players. Loaded: " + masterDeck.size() + " cards.");
        }

        Collections.shuffle(masterDeck, ThreadLocalRandom.current());
        List<Card> player1Deck = new ArrayList<>();
        List<Card> player2Deck = new ArrayList<>();

```



```

        for (int i = 0; i < cardsPerPlayer; i++) {
            player1Deck.add(masterDeck.get(i));
            player2Deck.add(masterDeck.get(i + cardsPerPlayer));
        }
        return List.of(player1Deck, player2Deck);
    }
}

```

```

public enum AIDifficulty {
    EASY, HARD
}

```

```

public interface AIStrategy {
    StatCategory selectStat(Card aiCard);
    String getRationale();
}

```

```

public static class MaxAdvantageStrategy implements AIStrategy {
    private String rationale;

```

@Override

```

    public StatCategory selectStat(Card aiCard) {
        StatCategory bestStat = null;
        double bestValue = Double.MIN_VALUE;

```

```

        for (StatCategory category : StatCategory.values()) {
            double currentValue = aiCard.getStat(category);

```

```

            double comparativeValue = category.isHigherIsBetter() ?
                currentValue : -currentValue;
            if (comparativeValue > bestValue) {
                bestStat = category;
                bestValue = comparativeValue;
            }
        }

```

```

        if (bestStat == null) {
            StatCategory[] categories = StatCategory.values();
            bestStat = categories[ThreadLocalRandom.current().nextInt(categories.length)];
        }
    }
}

```

```

        rationale = String.format("AI Hard chose %s (%.2f) as it provides the largest comparative
value on its card.",
            bestStat.getDisplayName(), aiCard.getStat(bestStat));
        return bestStat;
    }

```

```

    @Override
    public String getRationale() {
        return rationale;
    }
}

```

```

public static class RandomStrategy implements AIStrategy {
    private String rationale;

```

```

    @Override
    public StatCategory selectStat(Card aiCard) {
        StatCategory[] categories = StatCategory.values();
        StatCategory randomStat =
categories[ThreadLocalRandom.current().nextInt(categories.length)];
        rationale = String.format("AI Easy chose %s randomly.", randomStat.getDisplayName());
        return randomStat;
    }

```

```

    @Override
    public String getRationale() {
        return rationale;
    }
}

```

```

public static class AIContext {
    private AIStrategy strategy = new MaxAdvantageStrategy();

```

```

    public void setStrategy(AIStrategy strategy) {
        this.strategy = strategy;
    }

```

```

    public StatCategory executeStrategy(Card aiCard) {
        return strategy.selectStat(aiCard);
    }

```

```
    public String getRationale() {  
        return strategy.getRationale();  
    }  
}
```

```
public interface GameObserver {  
    void updateScore(int player1Score, int player2Score, int prizePileSize);  
    void updateTurn(int currentPlayer);  
    void updateCardDisplay(Card player1Card, Card player2Card, StatCategory challengedStat,  
String resultMessage);  
    void updateHistory(String entry);  
    void updateGameEnd(String winnerName, List<Card> winningDeck);  
    void updateStatusMessage(String message);  
    void updateHistoryReview(String fullHistory);  
}
```

```
public static abstract class GameNotifier {  
    private final List<GameObserver> observers = new ArrayList<>();  
  
    public void addObserver(GameObserver observer) {  
        observers.add(observer);  
    }  
}
```

```
    protected void notifyScoreUpdate(int p1, int p2, int prize) {  
        for (GameObserver obs : observers) {  
            obs.updateScore(p1, p2, prize);  
        }  
    }  
}
```

```
    protected void notifyTurnUpdate(int player) {  
        for (GameObserver obs : observers) {  
            obs.updateTurn(player);  
        }  
    }  
}
```

```
    protected void notifyCardDisplay(Card p1Card, Card p2Card, StatCategory stat, String  
result) {  
        for (GameObserver obs : observers) {  
            obs.updateCardDisplay(p1Card, p2Card, stat, result);  
        }  
    }  
}
```

```

    }
}

protected void notifyHistoryUpdate(String entry) {
    for (GameObserver obs : observers) {
        obs.updateHistory(entry);
    }
}

protected void notifyGameEnd(String winnerName, List<Card> winningDeck) {
    for (GameObserver obs : observers) {
        obs.updateGameEnd(winnerName, winningDeck);
    }
}

protected void notifyStatusMessage(String message) {
    for (GameObserver obs : observers) {
        obs.updateStatusMessage(message);
    }
}

protected void notifyHistoryReview(String fullHistory) {
    for (GameObserver obs : observers) {
        obs.updateHistoryReview(fullHistory);
    }
}
}

public interface TurnState {
    void handleStartTurn(GameEngine context);
    void handleStatSelection(GameEngine context, StatCategory stat);
    void handleResolution(GameEngine context);
    String getStatus();
}

public static class SelectionState implements TurnState {
    @Override
    public void handleStartTurn(GameEngine context) {
        context.notifyCardDisplay(context.getPlayer1CurrentCard(),

```

```

context.getPlayer2CurrentCard(), null, null);
    context.notifyTurnUpdate(context.getCurrentPlayer());
    context.notifyStatusMessage(context.getPlayerName(context.getCurrentPlayer()) + ":
Select a stat from your card.");

```

```

    if (context.getCurrentPlayer() == 2 && context.getGameMode() ==
GameMode.AIVSCOMPUTER) {
        context.notifyStatusMessage("PLAYER 2 AI is choosing stat...");

```

```

        javax.swing.Timer timer = new javax.swing.Timer(1000, new
java.awt.event.ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                StatCategory aiStat =
context.getAIContext().executeStrategy(context.getPlayer2CurrentCard());
                context.handleStatSelection(aiStat);
                javax.swing.Timer t = (javax.swing.Timer) e.getSource();
                t.stop();
                t.setRepeats(false);
            }
        });
        timer.setRepeats(false);
        timer.start();
    }
}

```

```

@Override
public void handleStatSelection(GameEngine context, StatCategory stat) {
    context.setChallengedStat(stat);
    context.setCurrentState(new ComparisonState());
    context.handleResolution();
}

```

```

@Override
public void handleResolution(GameEngine context) {

}

```

```

@Override
public String getStatus() {
    return "SELECTION";
}

```

```
}  
}
```

```
public static class ComparisonState implements TurnState {  
    @Override  
    public void handleStartTurn(GameEngine context) {  
        context.notifyStatusMessage("Comparison in progress...");  
    }  
}
```

```
@Override  
public void handleStatSelection(GameEngine context, StatCategory stat) {  
    context.notifyStatusMessage("Stat already selected.");  
}
```

```
@Override  
public void handleResolution(GameEngine context) {  
    StatCategory challengedStat = context.getChallengedStat();  
    Card p1Card = context.getPlayer1CurrentCard();  
    Card p2Card = context.getPlayer2CurrentCard();
```

```
    double p1Modifier = context.calculateMatchupModifier(p1Card, p2Card,  
challengedStat);  
    double p2Modifier = context.calculateMatchupModifier(p2Card, p1Card,  
challengedStat);
```

```
    double p1Value = p1Card.getStat(challengedStat) * (1 + p1Modifier);  
    double p2Value = p2Card.getStat(challengedStat) * (1 + p2Modifier);
```

```
    String matchupMessage = "";  
    if (p1Modifier != 0 || p2Modifier != 0) {  
        matchupMessage = String.format(" [MODIFIER: P1+%.1f%% vs P2+%.1f%%]",  
            p1Modifier * 100, p2Modifier * 100);  
    }  
}
```

```
    String resultMessage;  
    int winner = 0;  
    boolean higherIsBetter = challengedStat.isHigherIsBetter();
```

```

        if (higherIsBetter ? (p1Value > p2Value) : (p1Value < p2Value)) {
            winner = 1;
            resultMessage = String.format("%s Wins! %.2f vs %.2f%s",
                context.getPlayerName(1), p1Value, p2Value, matchupMessage);
        } else if (higherIsBetter ? (p2Value > p1Value) : (p2Value < p1Value)) {
            winner = 2;
            resultMessage = String.format("%s Wins! %.2f vs %.2f%s",
                context.getPlayerName(2), p2Value, p1Value, matchupMessage);
        } else {
            winner = 0;
            resultMessage = String.format("DRAW! %.2f vs %.2f%s", p1Value, p2Value,
matchupMessage);
        }

```

```

context.setRoundWinner(winner);
context.setRoundResultMessage(resultMessage);

```

```

context.notifyCardDisplay(p1Card, p2Card, challengedStat, resultMessage);

```

```

        javax.swing.Timer timer = new javax.swing.Timer(2000, new
java.awt.event.ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                context.setCurrentState(new ResolutionState());
                context.handleResolution();
                javax.swing.Timer t = (javax.swing.Timer) e.getSource();
                t.stop();
                t.setRepeats(false);
            }
        });
        timer.setRepeats(false);
        timer.start();
    }

```

```

    @Override
    public String getStatus() {
        return "COMPARISON";
    }
}

```

```

public static class ResolutionState implements TurnState {
    private int nextPlayer = 0;

    @Override
    public void handleStartTurn(GameEngine context) {
        context.notifyStatusMessage("Resolving turn outcome...");
    }

    @Override
    public void handleStatSelection(GameEngine context, StatCategory stat) {

    }

    @Override
    public void handleResolution(GameEngine context) {
        int winner = context.getRoundWinner();
        Card p1Card = context.getPlayer1CurrentCard();
        Card p2Card = context.getPlayer2CurrentCard();
        StatCategory challengedStat = context.getChallengedStat();
        String result = context.getRoundResultMessage();

        List<Card> prizeCards = new ArrayList<>();
        prizeCards.add(p1Card);
        prizeCards.add(p2Card);

        nextPlayer = context.getCurrentPlayer();

        if (winner != 0) {
            context.transferCards(winner, prizeCards);
            if (context.getPrizePileSize() > 0) {
                context.transferCards(winner, context.getPrizePile());
                context.notifyStatusMessage(String.format("%s takes the round AND the massive
Prize Pile!",
                    context.getPlayerName(winner)));
                context.clearPrizePile();
            }
            nextPlayer = winner;
        } else {

            context.addCardsToPrizePile(prizeCards);

```



```

        context.notifyStatusMessage(String.format("DRAW! Cards moved to the Prize Pile
(%d total cards). %s challenges again.",
            context.getPrizePileSize(), context.getPlayerName(context.getCurrentPlayer())));
    }

```

```

if (context.getPlayer1DeckSize() == 0 || context.getPlayer2DeckSize() == 0) {
    String winnerName;
    List<Card> winningDeck;

```

```

    if (context.getPlayer1DeckSize() == 0) {

        winnerName = context.getPlayerName(2);
        winningDeck = context.getPlayer2Deck();
    } else {

```

```

        winnerName = context.getPlayerName(1);
        winningDeck = context.getPlayer1Deck();
    }

```

```

    context.saveHighScore(winnerName, context.getTurnCount());
    context.notifyGameEnd(winnerName, winningDeck);
    context.saveMatchHistory();
    context.setCurrentState(new GameOverState());
    return;
}

```

```

context.setCurrentPlayer(nextPlayer);

```

```

context.prepareNextTurn();
context.setCurrentState(new SelectionState());
context.handleStartTurn();
}

```

```

@Override
public String getStatus() {
    return "RESOLUTION";
}

```

```
}
```

```
public static class GameOverState implements TurnState {  
    @Override  
    public void handleStartTurn(GameEngine context) {  
        context.notifyStatusMessage("Game Over. Click RESTART.");  
    }  
  
    @Override  
    public void handleStatSelection(GameEngine context, StatCategory stat) {  
        context.notifyStatusMessage("Game Over.");  
    }  
  
    @Override  
    public void handleResolution(GameEngine context) {  
        context.notifyStatusMessage("Game Over.");  
    }  
  
    @Override  
    public String getStatus() {  
        return "GAMEOVER";  
    }  
}
```

```
public static class GameEngine extends GameNotifier {  
    public void handleStartTurn() {  
        currentState.handleStartTurn(this);  
    }  
  
    private static GameEngine instance;  
    private final String HIGHSCOREFILE = "highscores.dat";  
    private final String MATCHHISTORYFILE = "matchhistory.dat";  
  
    private Queue<Card> player1Deck;  
    private Queue<Card> player2Deck;  
    private Card player1CurrentCard;  
    private Card player2CurrentCard;  
    private int currentPlayer;
```

```
private int turnCount;
private AIDifficulty difficulty = AIDifficulty.HARD;
private GameMode gameMode = GameMode.AIVSCOMPUTER;
private List<Card> prizePile = new ArrayList<>();
private List<String> currentMatchHistory = new ArrayList<>();
```

```
private String player1Name = "Player";
```

```
private TurnState currentState;
private AIContext aiContext;
```

```
private StatCategory challengedStat;
private int roundWinner;
private String roundResultMessage;
private List<ScoreEntry> highScores;
```

```
private GameEngine() {
    this.aiContext = new AIContext();
    this.highScores = loadHighScores();
    this.currentMatchHistory = new ArrayList<>();
    this.prizePile = new ArrayList<>();
    this.currentState = new SelectionState();
    this.currentPlayer = 1;
    this.turnCount = 0;
    notifyStatusMessage("Welcome! Click START GAME.");
}
```

```
public static GameEngine getInstance() {
    if (instance == null) {
        instance = new GameEngine();
    }
    return instance;
}
```

```
public void setPlayer1Name(String name) {
    if (name == null || name.trim().isEmpty()) {
        this.player1Name = "Player";
    } else {
```

```

        this.player1Name = name.trim();
    }
}

```

```

public String getPlayerName(int player) {
    if (player == 1) return player1Name;
    return "AI - " + difficulty.toString();
}

```

```

public void addCleanWinnerHistory(String winner) {
    currentMatchHistory.add(winner + " won");
}

```

```

public void startGame() {
    try {
        DeckManager deckManager = new DeckManager();

        List<List<Card>> decks = deckManager.dealCards(11);
        player1Deck = new LinkedList<>(decks.get(0));
        player2Deck = new LinkedList<>(decks.get(1));

```

```

        this.turnCount = 0;
        this.currentPlayer = 1;
        this.currentMatchHistory.clear();
        this.prizePile.clear();

```

```

        aiContext.setStrategy(difficulty == AIDifficulty.HARD ?
            new MaxAdvantageStrategy() : new RandomStrategy());
        prepareNextTurn();
        setCurrentState(new SelectionState());
        currentState.handleStartTurn(this);

```

```

        notifyScoreUpdate(player1Deck.size(), player2Deck.size(), prizePile.size());
        notifyTurnUpdate(currentPlayer);

```

```

        notifyHistoryUpdate("--- New Game Started in " + gameMode.toString() + " Mode - 11
Cards Each ---");
    } catch (IllegalStateException e) {
        notifyStatusMessage("ERROR: Not enough players to start the game. " +

```

```
e.getMessage());
    setCurrentState(new GameOverState());
}
}
```

```
public void endGame() {
    if (player1Deck == null || currentState instanceof GameOverState) {
        notifyStatusMessage("No match is currently running to end.");
        setCurrentState(new GameOverState());
        return;
    }
}
```

```
int p1TotalCards = player1Deck.size();
int p2TotalCards = player2Deck.size();
String winnerName;
List<Card> winningDeck;
```

```
if (p1TotalCards > p2TotalCards) {
    winnerName = getPlayerName(1);
    winningDeck = new ArrayList<>(player1Deck);
} else if (p2TotalCards > p1TotalCards) {
    winnerName = getPlayerName(2);
    winningDeck = new ArrayList<>(player2Deck);
} else {
    winnerName = "Abrupt End - Draw";
    winningDeck = Collections.emptyList();
}
```

```
saveHighScore(winnerName, turnCount);
```

```
player1CurrentCard = null;
player2CurrentCard = null;
prizePile.clear();
```

```
notifyGameEnd(winnerName, winningDeck);
saveMatchHistory();
setCurrentState(new GameOverState());
}
```

```

public double calculateMatchupModifier(Card attacker, Card defender, StatCategory stat)
{
    double modifier = 0.0;
    PlayerType attackerType = attacker.getPlayerType();

    if (stat == StatCategory.BATTINGAVG || stat == StatCategory.STRIKERATE) {
        if (attackerType == PlayerType.BATSMAN ||
            attackerType == PlayerType.WICKETKEEPER ||
            attackerType == PlayerType.ALLROUNDER) {
            modifier = 0.25;
        }
    }

    else if (stat == StatCategory.BOWLINGWICKETS || stat == StatCategory.ECONOMY) {
        if (attackerType == PlayerType.BOWLER || attackerType == PlayerType.ALLROUNDER)
        {
            modifier = 0.25;
        }
    }

    return modifier;
}

public void setCurrentState(TurnState newState) {
    this.currentState = newState;
}

public void handleStatSelection(StatCategory stat) {
    if (currentState.getStatus().equals("SELECTION")) {
        currentState.handleStatSelection(this, stat);
    } else {
        notifyStatusMessage("Wait for the current round to resolve before selecting a stat.");
    }
}
}

```

```
public void handleResolution() {  
    currentState.handleResolution(this);  
}
```

```
public void prepareNextTurn() {  
    turnCount++;  
  
    player1CurrentCard = player1Deck.poll();  
    player2CurrentCard = player2Deck.poll();  
    notifyScoreUpdate(player1Deck.size(), player2Deck.size(), prizePile.size());  
}
```

```
public void transferCards(int winner, List<Card> cardsToTransfer) {  
    Queue<Card> winnerDeck = (winner == 1) ? player1Deck : player2Deck;  
  
    winnerDeck.addAll(cardsToTransfer);  
  
    notifyScoreUpdate(player1Deck.size(), player2Deck.size(), prizePile.size());  
}
```

```
public void addCardsToPrizePile(List<Card> cards) {  
    prizePile.addAll(cards);  
    notifyScoreUpdate(player1Deck.size(), player2Deck.size(), prizePile.size());  
}
```

```
public void clearPrizePile() {  
    prizePile.clear();  
    notifyScoreUpdate(player1Deck.size(), player2Deck.size(), prizePile.size());  
}
```

```
public Card getPlayer1CurrentCard() { return player1CurrentCard; }  
public Card getPlayer2CurrentCard() { return player2CurrentCard; }  
public int getCurrentPlayer() { return currentPlayer; }  
public void setCurrentPlayer(int currentPlayer) { this.currentPlayer = currentPlayer; }  
public StatCategory getChallengedStat() { return challengedStat; }  
public void setChallengedStat(StatCategory challengedStat) { this.challengedStat =  
challengedStat; }  
public int getRoundWinner() { return roundWinner; }  
public void setRoundWinner(int roundWinner) { this.roundWinner = roundWinner; }
```

```

    public String getRoundResultMessage() { return roundResultMessage; }
    public void setRoundResultMessage(String roundResultMessage) {
this.roundResultMessage = roundResultMessage; }
    public int getTurnCount() { return turnCount; }
    public GameMode getGameMode() { return gameMode; }
    public AIDifficulty getAIDifficulty() { return difficulty; }
    public AIContext getAIContext() { return aiContext; }
    public int getPlayer1DeckSize() { return player1Deck.size(); }
    public int getPlayer2DeckSize() { return player2Deck.size(); }
    public List<Card> getPrizePile() { return prizePile; }
    public int getPrizePileSize() { return prizePile.size(); }
    public List<Card> getPlayer1Deck() { return new ArrayList<>(player1Deck); }
    public List<Card> getPlayer2Deck() { return new ArrayList<>(player2Deck); }
    public List<ScoreEntry> getHighScores() { return highScores; }

```

```

    public void setAIDifficulty(AIDifficulty difficulty) {
        this.difficulty = difficulty;

        aiContext.setStrategy(difficulty == AIDifficulty.HARD ?
            new MaxAdvantageStrategy() : new RandomStrategy());
        notifyStatusMessage("AI Difficulty set to " + difficulty.toString());
    }

```

```

    public static class ScoreEntry implements Serializable, Comparable<ScoreEntry> {
        String winnerName;
        int turnsTaken;
        String mode;

        public ScoreEntry(String winnerName, int turnsTaken, String mode) {
            this.winnerName = winnerName;
            this.turnsTaken = turnsTaken;
            this.mode = mode;
        }
    }

```

```

    @Override
    public String toString() {
        return String.format("%s - %s (%d turns)", winnerName, mode, turnsTaken);
    }

```

```

    @Override

```



```

    public int compareTo(ScoreEntry other) {
        return Integer.compare(this.turnsTaken, other.turnsTaken);
    }
}

```

```

    public void saveHighScore(String winnerName, int turnsTaken) {
        ScoreEntry newScore = new ScoreEntry(winnerName, turnsTaken,
gameMode.toString());
        highScores.add(newScore);

        Collections.sort(highScores);
        if (highScores.size() > 10) {
            highScores = highScores.subList(0, 10);
        }
        saveHighScores();
    }

```

```

    private void saveHighScores() {
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream(HIGHSCOREFILE))) {
            oos.writeObject(highScores);
            System.out.println("High scores saved successfully.");
        } catch (IOException e) {
            System.err.println("Error saving high scores: " + e.getMessage());
        }
    }

```

```

@SuppressWarnings("unchecked")
    private List<ScoreEntry> loadHighScores() {
        File file = new File(HIGHSCOREFILE);
        if (file.exists()) {
            try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(file))) {
                List<ScoreEntry> loadedScores = (List<ScoreEntry>) ois.readObject();
                System.out.println("High scores loaded successfully (" + loadedScores.size() + "
entries).");
                return loadedScores;
            } catch (IOException | ClassNotFoundException e) {
                System.err.println("Error loading high scores: " + e.getMessage());
            }
        }
        return new ArrayList<>();
    }

```

```

    }

    public void saveMatchHistory() {

    }

    public String getFullHistoryReview() {

        StringBuilder sb = new StringBuilder();
        sb.append("--- HIGH SCORES (Lowest Turns Win) ---\n");
        if (highScores.isEmpty()) {
            sb.append("No high scores recorded yet.\n");
        } else {
            for (int i = 0; i < highScores.size(); i++) {
                ScoreEntry entry = highScores.get(i);
                sb.append(String.format("%d. %s\n", i + 1, entry.toString()));
            }
        }
        sb.append("\n--- MATCH HISTORY (Current Session) ---\n");
        if (currentMatchHistory.isEmpty()) {
            sb.append("No games finished this session.\n");
        } else {
            for (String entry : currentMatchHistory) {
                sb.append(entry).append("\n");
            }
        }
        return sb.toString();
    }
}

```

```

private static final Color DARK_BG = new Color(10, 20, 30);
private static final Color ACCENT_GREEN = new Color(50, 205, 50);
private static final Color ACCENT_GOLD = new Color(255, 215, 0);

```

```

static class GlassPanel extends JPanel {

```

```

public GlassPanel(LayoutManager layout) {
    super(layout);
    setOpaque(false);
    setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
}

```

```

@Override
protected void paintComponent(Graphics g) {
    Graphics2D g2d = (Graphics2D) g.create();
    g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

    g2d.setColor(new Color(0, 0, 0, 150));
    g2d.fillRoundRect(0, 0, getWidth(), getHeight(), 25, 25);

    g2d.setColor(new Color(255, 255, 255, 50));
    g2d.drawRoundRect(0, 0, getWidth() - 1, getHeight() - 1, 25, 25);
    g2d.dispose();
    super.paintComponent(g);
}
}

```

```

static class ModernButton extends JButton {
    public ModernButton(String text, int width, int height) {
        super(text);
        setPreferredSize(new Dimension(width, height));
        setFont(new Font("Segoe UI", Font.BOLD, 18));
        setBackground(ACCENT_GREEN);
        setForeground(Color.WHITE);
        setFocusPainted(false);
        setBorder(BorderFactory.createEmptyBorder(10, 15, 10, 15));
    }
}

```

```

static class HeroButton extends JButton {
    public HeroButton(String text, int width, int height) {
        super(text);
        setPreferredSize(new Dimension(width, height));
        setFont(new Font("Segoe UI", Font.BOLD, 24));
    }
}

```

```

        setBackground(ACCENT_GOLD);
        setForeground(DARK_BG);
        setFocusPainted(false);
        setBorder(BorderFactory.createEmptyBorder(15, 20, 15, 20));
    }
}

```

```

static class StatButton extends JButton {
    private StatCategory category;
    private boolean isSelected = false;

    public StatButton(StatCategory category) {
        this.category = category;
        setText(category.getDisplayName());
        setFont(new Font("Segoe UI", Font.BOLD, 18));
        setForeground(Color.WHITE);
        setFocusPainted(false);
        setContentAreaFilled(false);
        setBorder(BorderFactory.createEmptyBorder(15, 15, 15, 15));

        setName(category.name());
    }

    public StatCategory getCategory() {
        return category;
    }

    public void setSelected(boolean isSelected) {
        this.isSelected = isSelected;
        repaint();
    }

    @Override
    protected void paintComponent(Graphics g) {
        Graphics2D g2d = (Graphics2D) g.create();
        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

        int w = getWidth();
        int h = getHeight();

```

```

    if (isSelected) {
        g2d.setColor(ACCENT_GOLD);
        g2d.fillRoundRect(0, 0, w, h, 20, 20);
    } else {

        g2d.setColor(new Color(30, 40, 50));
        g2d.fillRoundRect(0, 0, w, h, 20, 20);

        if (getModel().isRollover() || getModel().isPressed()) {
            g2d.setColor(new Color(50, 60, 70, 200));
            g2d.fillRoundRect(0, 0, w, h, 20, 20);
        }
    }

    g2d.dispose();
    super.paintComponent(g);
}
}

```

```

public static class CricketAttaxEliteGUI extends JFrame implements GameObserver {
    private GameEngine engine = GameEngine.getInstance();
    private CardLayout cardLayout;
    private GlassBackgroundPanel mainContainer;
    private StartScreenPanel startScreen;
    private GameScreenPanel gameScreen;
    private EndScreenPanel endScreen;

    private JLabel scoreLabel;
    private JLabel turnLabel;
    private JTextArea statusMessageArea;
    private CardDisplayPanel player1CardDisplay;
    private CardDisplayPanel player2CardDisplay;
    private StatSelectionPanel statSelectionPanel;
    // Removed historyPanel (Recent Matches)

```

```

public CricketAttaxEliteGUI() {

```

```

super("🏏 CRICKET ATTAX ELITE - PROFESSIONAL EDITION 🏏");
initProfessionalUI();

engine.addObserver(this);
setVisible(true);
}

private void initProfessionalUI() {
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setSize(1400, 900);
    setMinimumSize(new Dimension(1200, 800));
    setLocationRelativeTo(null);
    getContentPane().setBackground(DARK_BG);

    setUndecorated(true);
    add(createTitleBar(), BorderLayout.NORTH);

    cardLayout = new CardLayout();
    mainContainer = new GlassBackgroundPanel();
    mainContainer.setLayout(cardLayout);
    add(mainContainer, BorderLayout.CENTER);

    createAllScreens();
    cardLayout.show(mainContainer, "START");
}

private JPanel createTitleBar() {
    JPanel titleBar = new JPanel(new BorderLayout());
    titleBar.setOpaque(false);
    titleBar.setPreferredSize(new Dimension(0, 60));
    titleBar.setBorder(BorderFactory.createEmptyBorder(0, 10, 0, 10));

    JLabel title = new JLabel("🏏 CRICKET ATTAX ELITE", SwingConstants.CENTER);
    title.setFont(new Font("Segoe UI", Font.BOLD, 20));
    title.setForeground(Color.WHITE);
    titleBar.add(title, BorderLayout.CENTER);
}

```

```
JPanel controls = new JPanel(new FlowLayout(FlowLayout.RIGHT));
controls.setOpaque(false);
```

```
JButton minimizeBtn = new ModernButton("—", 40, 40);
minimizeBtn.addActionListener(e -> setState(JFrame.ICONIFIED));
```

```
JButton closeBtn = new ModernButton("✖", 40, 40);
closeBtn.setBackground(new Color(200, 50, 50));
```

```
closeBtn.setToolTipText("Close application (asks confirmation)");
closeBtn.addActionListener(e -> {
    int confirm = JOptionPane.showConfirmDialog(this,
        "Close the application? This will quit immediately and WILL NOT save the current
match. Continue?",
        "Close Application", JOptionPane.YES_NO_OPTION,
JOptionPane.WARNING_MESSAGE);
    if (confirm == JOptionPane.YES_OPTION) {
        System.exit(0);
    }
});
```

```
JButton exitNoSaveBtn = new ModernButton("✖ Exit (No Save)", 160, 40);
exitNoSaveBtn.setBackground(new Color(180, 30, 30));
exitNoSaveBtn.setToolTipText("Exit immediately without saving or modifying the
current match");
exitNoSaveBtn.addActionListener(e -> {
    int confirm = JOptionPane.showConfirmDialog(this,
        "Exit now? This will quit immediately and WILL NOT save or alter the current
match. Continue?",
        "Confirm Exit (No Save)", JOptionPane.YES_NO_OPTION,
JOptionPane.WARNING_MESSAGE);
    if (confirm == JOptionPane.YES_OPTION) {
        System.exit(0);
    }
});
```

```
controls.add(minimizeBtn);
controls.add(exitNoSaveBtn);
```

```

controls.add(closeBtn);
titleBar.add(controls, BorderLayout.EAST);

final Point offset = new Point();
titleBar.addMouseListener(new MouseAdapter() {
    @Override
    public void mousePressed(MouseEvent e) {
        offset.setLocation(e.getX(), e.getY());
    }
});
titleBar.addMouseMotionListener(new MouseAdapter() {
    @Override
    public void mouseDragged(MouseEvent e) {
        Point currentScreenLocation = getLocationOnScreen();
        setLocation(currentScreenLocation.x - offset.x + e.getX(),
            currentScreenLocation.y - offset.y + e.getY());
    }
});
return titleBar;
}

private void createAllScreens() {
    startScreen = new StartScreenPanel(this);
    gameScreen = new GameScreenPanel(this);
    endScreen = new EndScreenPanel(this);

    mainContainer.add(startScreen, "START");
    mainContainer.add(gameScreen, "GAME");
    mainContainer.add(endScreen, "END");

    this.scoreLabel = gameScreen.scoreLabel;
    this.turnLabel = gameScreen.turnLabel;
    this.statusMessageArea = gameScreen.statusArea;
    this.player1CardDisplay = gameScreen.player1Display;
    this.player2CardDisplay = gameScreen.player2Display;
    this.statSelectionPanel = gameScreen.statPanel;
    // historyPanel removed
}

```



```

public void startGame(String playerName, AIDifficulty difficulty) {
    engine.setPlayer1Name(playerName);
    engine.setAIDifficulty(difficulty);
    engine.startGame();
    cardLayout.show(mainContainer, "GAME");
}

```

```

public void showEndGame(String winnerName, List<GameEngine.ScoreEntry> scores) {
    endScreen.updateWinner(winnerName, scores);
    cardLayout.show(mainContainer, "END");
}

```

@Override

```

public void updateScore(int player1Score, int player2Score, int prizePileSize) {
    String prizeText = prizePileSize > 0 ? String.format(" | Prize Pile: %d", prizePileSize) : "";
    scoreLabel.setText(String.format("Score: %s %d - %d %s%s",
        engine.getPlayerName(1), player1Score, player2Score, engine.getPlayerName(2),
prizeText));
}

```

@Override

```

public void updateTurn(int currentPlayer) {
    String name = engine.getPlayerName(currentPlayer);
    if (currentPlayer == 1) {
        turnLabel.setText("👤 YOUR TURN - Choose Wisely");
        turnLabel.setForeground(ACCENT_GOLD);
        statSelectionPanel.enableSelection();
        player1CardDisplay.updateCard(engine.getPlayer1CurrentCard());
        player2CardDisplay.updateCard(null);
    } else {
        turnLabel.setText("🤖 AI's TURN - Waiting for selection...");
        turnLabel.setForeground(ACCENT_GREEN);
        statSelectionPanel.disableSelection();
        player1CardDisplay.updateCard(engine.getPlayer1CurrentCard());
        player2CardDisplay.updateCard(null);
    }
}

```

```

@Override
public void updateCardDisplay(Card player1Card, Card player2Card, StatCategory
challengedStat, String resultMessage) {
    player1CardDisplay.updateCard(player1Card);
    player2CardDisplay.updateCard(player2Card);

    if (player2Card != null) {

        statSelectionPanel.highlightStat(challengedStat);
        statSelectionPanel.disableSelection();
        if (resultMessage != null) {
            statusMessageArea.setText(resultMessage + "\n" +
engine.getAIContext().getRationale());
        }
    } else {

        statSelectionPanel.clearHighlight();
        statusMessageArea.setText("");
    }
}

```

```

@Override
public void updateHistory(String entry) {

}

```

```

@Override
public void updateGameEnd(String winnerName, List<Card> winningDeck) {
    engine.addCleanWinnerHistory(winnerName);
    updateScore(winningDeck.size(), 0, 0);
    showEndGame(winnerName, engine.getHighScores());
}

```

```

@Override
public void updateStatusMessage(String message) {
    statusMessageArea.setText(message);
}

```

```

@Override
public void updateHistoryReview(String fullHistory) {

```

```
}  
}
```

```
static class GlassBackgroundPanel extends JPanel {  
    private BufferedImage stadiumGradient = createStadiumGradient();  
  
    public GlassBackgroundPanel() {  
        setOpaque(true);  
    }  
  
    private BufferedImage createStadiumGradient() {  
        BufferedImage img = new BufferedImage(1400, 900, BufferedImage.TYPE_INT_ARGB);  
        Graphics2D g2d = img.createGraphics();  
  
        GradientPaint greenGradient = new GradientPaint(  
            0, 0, new Color(34, 139, 34),  
            1400, 900, new Color(0, 100, 0)  
        );  
        g2d.setPaint(greenGradient);  
        g2d.fillRect(0, 0, 1400, 900);  
  
        g2d.setComposite(AlphaComposite.getInstance(AlphaComposite.SRC_OVER, 0.3f));  
        g2d.setColor(Color.WHITE);  
        g2d.fillOval(100, 100, 200, 200);  
        g2d.fillOval(1100, 100, 200, 200);  
        g2d.dispose();  
        return img;  
    }  
  
    @Override  
    protected void paintComponent(Graphics g) {  
        super.paintComponent(g);  
  
        if (stadiumGradient != null) {  
            g.drawImage(stadiumGradient, 0, 0, getWidth(), getHeight(), this);  
        }  
    }  
}
```

```
}
```

```
static class StartScreenPanel extends JPanel {  
    private final CricketAttaxEliteGUI parent;
```

```
    public StartScreenPanel(CricketAttaxEliteGUI parent) {  
        this.parent = parent;  
        setOpaque(false);  
        setLayout(new BoxLayout(this, BoxLayout.Y_AXIS));  
        setBorder(BorderFactory.createEmptyBorder(80, 40, 80, 40));  
        createHeroStartScreen();  
    }
```

```
    private void createHeroStartScreen() {
```

```
        JLabel heroTitle = new JLabel("🏏 CRICKET ATTAX ELITE", SwingConstants.CENTER);  
        heroTitle.setFont(new Font("Segoe UI", Font.BOLD, 64));  
        heroTitle.setForeground(Color.WHITE);  
        heroTitle.setAlignmentX(Component.CENTER_ALIGNMENT);  
        add(heroTitle);  
        add(Box.createVerticalStrut(20));
```

```
        JLabel subtitle = new JLabel("PROFESSIONAL CRICKET CARD BATTLES",  
SwingConstants.CENTER);  
        subtitle.setFont(new Font("Segoe UI", Font.BOLD, 24));  
        subtitle.setForeground(new Color(200, 255, 200));  
        subtitle.setAlignmentX(Component.CENTER_ALIGNMENT);  
        add(subtitle);  
        add(Box.createVerticalStrut(60));
```

```
        JPanel settingsContainer = new JPanel(new GridLayout(1, 2, 30, 0));  
        settingsContainer.setOpaque(false);  
        settingsContainer.setAlignmentX(Component.CENTER_ALIGNMENT);
```

```
        GameEngine engine = GameEngine.getInstance();
```

```

GlassPanel playerCard = new GlassPanel(new BorderLayout(10, 10));
playerCard.setBorder(BorderFactory.createTitledBorder(
    BorderFactory.createLineBorder(ACCENT_GOLD, 1),
    "Your Profile", TitledBorder.LEFT, TitledBorder.TOP,
    new Font("Segoe UI", Font.BOLD, 16), ACCENT_GOLD));
JTextField nameField = new JTextField(engine.getPlayerName(1), 20);
nameField.setFont(new Font("Segoe UI", Font.PLAIN, 18));
playerCard.add(new JLabel("Player Name:"), BorderLayout.NORTH);
playerCard.add(nameField, BorderLayout.CENTER);
settingsContainer.add(playerCard);

```

```

GlassPanel difficultyCard = new GlassPanel(new BorderLayout(10, 10));
difficultyCard.setBorder(BorderFactory.createTitledBorder(
    BorderFactory.createLineBorder(ACCENT_GREEN, 1),
    "Game Settings", TitledBorder.LEFT, TitledBorder.TOP,
    new Font("Segoe UI", Font.BOLD, 16), ACCENT_GREEN));
JComboBox<AIDifficulty> diffCombo = new JComboBox<>(AIDifficulty.values());
diffCombo.setSelectedItem(engine.getAIDifficulty());
diffCombo.setFont(new Font("Segoe UI", Font.PLAIN, 18));
difficultyCard.add(new JLabel("AI Difficulty:"), BorderLayout.NORTH);
difficultyCard.add(diffCombo, BorderLayout.CENTER);
settingsContainer.add(difficultyCard);

```

```

add(settingsContainer);
add(Box.createVerticalGlue());

```

```

HeroButton startBtn = new HeroButton("🚀 START LEGENDARY BATTLE", 400, 80);
startBtn.setAlignmentX(Component.CENTER_ALIGNMENT);
startBtn.addActionListener(e -> parent.startGame(nameField.getText(), (AIDifficulty)
diffCombo.getSelectedItem()));
    add(startBtn);
}
}

```

```

static class GameScreenPanel extends JPanel {
    private final CricketAttaxEliteGUI parent;

```

```

private JLabel scoreLabel;
private JLabel turnLabel;
private JTextArea statusArea;
private CardDisplayPanel player1Display;
private CardDisplayPanel player2Display;
private StatSelectionPanel statPanel;
private HistoryPanel historyPanel;

public GameScreenPanel(CricketAttaxEliteGUI parent) {
    this.parent = parent;
    setOpaque(false);
    setLayout(new BorderLayout(30, 30));
    setBorder(BorderFactory.createEmptyBorder(40, 40, 40, 40));
    createProfessionalGameLayout();
}

private void createProfessionalGameLayout() {

    add(createStatusBar(), BorderLayout.NORTH);

    JPanel centerPanel = new JPanel(new BorderLayout(40, 0));
    centerPanel.setOpaque(false);

    JPanel cardsArea = new JPanel(new GridLayout(1, 2, 40, 0));
    cardsArea.setOpaque(false);
    player1Display = new CardDisplayPanel(true);
    player2Display = new CardDisplayPanel(false);
    cardsArea.add(player1Display);
    cardsArea.add(player2Display);
    centerPanel.add(cardsArea, BorderLayout.CENTER);

    statPanel = new StatSelectionPanel(parent);
    centerPanel.add(statPanel, BorderLayout.EAST);
    add(centerPanel, BorderLayout.CENTER);
}

```

```
    add(createBottomPanel(), BorderLayout.SOUTH);  
}
```

```
private JPanel createStatusBar() {  
    GlassPanel statusBar = new GlassPanel(new BorderLayout(20, 0));  
    statusBar.setPreferredSize(new Dimension(0, 80));
```

```
    scoreLabel = new JLabel("Score: Player 0 - 0 AI", SwingConstants.CENTER);  
    scoreLabel.setFont(new Font("Segoe UI", Font.BOLD, 28));  
    scoreLabel.setForeground(ACCENT_GOLD);  
    statusBar.add(scoreLabel, BorderLayout.CENTER);
```

```
    turnLabel = new JLabel("👤 YOUR TURN - Choose Wisely", SwingConstants.CENTER);  
    turnLabel.setFont(new Font("Segoe UI", Font.BOLD, 20));  
    turnLabel.setForeground(Color.WHITE);  
    statusBar.add(turnLabel, BorderLayout.SOUTH);
```

```
    return statusBar;  
}
```

```
private JPanel createBottomPanel() {  
    JPanel bottom = new JPanel(new BorderLayout(20, 20));  
    bottom.setOpaque(false);
```

```
    GlassPanel statusPanel = new GlassPanel(new BorderLayout());  
    statusPanel.setPreferredSize(new Dimension(400, 180));  
    statusArea = new JTextArea();  
    statusArea.setEditable(false);  
    statusArea.setFont(new Font("Segoe UI", Font.PLAIN, 16));  
    statusArea.setBackground(new Color(0, 0, 0, 80));  
    statusArea.setForeground(Color.WHITE);  
    statusArea.setLineWrap(true);  
    statusArea.setWrapStyleWord(true);  
    statusArea.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));  
    statusPanel.add(new JScrollPane(statusArea), BorderLayout.CENTER);  
    statusPanel.setBorder(BorderFactory.createTitledBorder(  
        BorderFactory.createLineBorder(Color.LIGHT_GRAY, 1),
```

```
"Game Status / AI Rationale", TitledBorder.LEFT, TitledBorder.TOP,  
new Font("Segoe UI", Font.BOLD, 14), Color.LIGHT_GRAY));
```

```
bottom.add(statusPanel, BorderLayout.WEST);
```

```
JPanel rightControls = new JPanel(new BorderLayout(10, 10));  
rightControls.setOpaque(false);  
// Removed historyPanel from rightControls
```

```
JPanel actionButtons = new JPanel(new FlowLayout(FlowLayout.CENTER, 15, 0));  
actionButtons.setOpaque(false);
```

```
JButton endMatchButton = new ModernButton("End Match", 120, 40);  
endMatchButton.setBackground(new Color(150, 50, 50));  
endMatchButton.addActionListener(e -> GameEngine.getInstance().endGame());
```

```
JButton exitButton = new ModernButton("Exit Game", 120, 40);  
exitButton.setBackground(new Color(200, 50, 50));
```

```
exitButton.setToolTipText("Quit immediately (no save). Exits the application without  
modifying the current match.");  
exitButton.addActionListener(e -> {  
    int confirm = JOptionPane.showConfirmDialog(parent,  
        "Exit now? This will quit immediately and WILL NOT save or alter the current  
match. Continue?",  
        "Confirm Exit (No Save)", JOptionPane.YES_NO_OPTION,  
JOptionPane.WARNING_MESSAGE);  
    if (confirm == JOptionPane.YES_OPTION) {  
  
        System.exit(0);  
    }  
});
```

```
actionButtons.add(endMatchButton);  
actionButtons.add(exitButton);  
rightControls.add(actionButtons, BorderLayout.SOUTH);
```

```
bottom.add(rightControls, BorderLayout.CENTER);
```



```
        return bottom;
    }
}
```

```
static class CardDisplayPanel extends GlassPanel {
    private Card currentCard;
    private JLabel nameLabel;
    private StatsDisplay statsDisplay;
    private final boolean isPlayerCard;

    public CardDisplayPanel(boolean isPlayerCard) {
        super(new BorderLayout(10, 10));
        this.isPlayerCard = isPlayerCard;
        setPreferredSize(new Dimension(350, 0));
        setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));
        createCardLayout();
    }

    private void createCardLayout() {
        GlassPanel cardFrame = new GlassPanel(new BorderLayout());
        cardFrame.setBorder(BorderFactory.createLineBorder(ACCENT_GOLD, 3));
        cardFrame.setBackground(new Color(0, 0, 0, 180));

        JPanel header = new JPanel(new BorderLayout());
        header.setOpaque(false);
        header.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

        nameLabel = new JLabel("No Card", SwingConstants.CENTER);
        nameLabel.setFont(new Font("Segoe UI", Font.BOLD, 24));
        nameLabel.setForeground(ACCENT_GOLD);
        header.add(nameLabel, BorderLayout.CENTER);

        cardFrame.add(header, BorderLayout.NORTH);

        statsDisplay = new StatsDisplay();
        cardFrame.add(statsDisplay, BorderLayout.CENTER);
    }
}
```

```

        add(cardFrame, BorderLayout.CENTER);
    }

    public void updateCard(Card card) {
        this.currentCard = card;
        if (card != null) {

            nameLabel.setText(card.getPlayerName() + " (" + card.getPlayerType() + ")");

            statsDisplay.updateStats(card.getAllStats(), isPlayerCard);
        } else {
            nameLabel.setText(isPlayerCard ? "Your Deck" : "AI's Deck");
            statsDisplay.clear();
        }
        repaint();
    }
}

```

```

static class StatsDisplay extends JPanel {
    private final Map<StatCategory, JLabel> statLabels = new HashMap<>();

    public StatsDisplay() {
        setOpaque(false);
        setLayout(new BoxLayout(this, BoxLayout.Y_AXIS));
        createStatRows();
    }

    private void createStatRows() {
        for (StatCategory stat : StatCategory.values()) {
            JPanel row = new JPanel(new BorderLayout());
            row.setOpaque(false);
            row.setBorder(BorderFactory.createEmptyBorder(8, 20, 8, 20));
            row.setMaximumSize(new Dimension(Integer.MAX_VALUE, 45));

            JLabel statName = new JLabel(stat.getDisplayName(), SwingConstants.LEFT);
            statName.setFont(new Font("Segoe UI", Font.PLAIN, 16));
            statName.setForeground(new Color(220, 220, 220));

```

```

        JLabel valueLabel = new JLabel("—.--", SwingConstants.RIGHT);
        valueLabel.setFont(new Font("Segoe UI", Font.BOLD, 18));
        valueLabel.setForeground(ACCENT_GOLD);
        statLabels.put(stat, valueLabel);

        row.add(statName, BorderLayout.WEST);
        row.add(valueLabel, BorderLayout.EAST);
        add(row);
    }
}

public void updateStats(Map<StatCategory, Double> stats, boolean showAll) {
    for (StatCategory stat : StatCategory.values()) {
        Double value = stats.getOrDefault(stat, 0.0);
        JLabel label = statLabels.get(stat);
        label.setText(String.format("%.1f", value));

        if (showAll) {
            label.setVisible(true);
        } else {
            label.setVisible(false);
        }
    }
}

public void clear() {
    statLabels.values().forEach(label -> {
        label.setText("—.--");
        label.setVisible(true);
    });
}
}

static class StatSelectionPanel extends JPanel {
    private final CricketAttaxEliteGUI parent;
    private final Map<StatCategory, StatButton> statButtons = new HashMap<>();
    private boolean isEnabled = true;

    public StatSelectionPanel(CricketAttaxEliteGUI parent) {

```

```

    this.parent = parent;
    setOpaque(false);
    setPreferredSize(new Dimension(300, 0));
    setLayout(new GridLayout(StatCategory.values().length, 1, 0, 15));
    setBorder(BorderFactory.createEmptyBorder(50, 0, 50, 0));
    createStatButtons();
}

```

```

private void createStatButtons() {
    for (StatCategory stat : StatCategory.values()) {
        StatButton button = new StatButton(stat);
        button.addActionListener(e -> {
            if (isEnabled) {
                GameEngine.getInstance().handleStatSelection(stat);
                disableSelection();
            }
        });
        statButtons.put(stat, button);
        add(button);
    }
}

```

```

public void enableSelection() {
    isEnabled = true;
    for (StatButton button : statButtons.values()) {
        button.setEnabled(true);
        button.setSelected(false);
    }
}

```

```

public void disableSelection() {
    isEnabled = false;
    for (StatButton button : statButtons.values()) {
        button.setEnabled(false);
    }
}

```

```

public void highlightStat(StatCategory stat) {
    clearHighlight();
    if (stat != null) {
        statButtons.get(stat).setSelected(true);
    }
}

```

```

    }
}

public void clearHighlight() {
    for (StatButton button : statButtons.values()) {
        button.setSelected(false);
    }
}
}

static class HistoryPanel extends GlassPanel {
    private final DefaultListModel<String> historyModel;
    private final JList<String> historyList;

    public HistoryPanel() {
        super(new BorderLayout());
        setPreferredSize(new Dimension(0, 180));
        setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));

        historyModel = new DefaultListModel<>();
        historyList = new JList<>(historyModel);
        historyList.setFont(new Font("Segoe UI", Font.PLAIN, 14));
        historyList.setBackground(new Color(0, 0, 0, 120));
        historyList.setForeground(Color.WHITE);

        add(new JScrollPane(historyList), BorderLayout.CENTER);

        setBorder(BorderFactory.createTitledBorder(
            BorderFactory.createLineBorder(Color.LIGHT_GRAY, 1),
            "Recent Matches", TitledBorder.LEFT, TitledBorder.TOP,
            new Font("Segoe UI", Font.BOLD, 14), Color.LIGHT_GRAY));
    }

    public void updateHistory(List<String> history) {
        historyModel.clear();
        history.stream().limit(8).forEach(historyModel::addElement);
    }
}

```

```

static class EndScreenPanel extends JPanel {
    private final CricketAttaxEliteGUI parent;
    private JLabel victoryLabel;
    private JLabel highScoresLabel;
    private AnimatedTrophy trophy;

    public EndScreenPanel(CricketAttaxEliteGUI parent) {
        this.parent = parent;
        setOpaque(false);
        setLayout(new BoxLayout(this, BoxLayout.Y_AXIS));
        createVictoryScreen();
    }

    private void createVictoryScreen() {
        setBorder(BorderFactory.createEmptyBorder(80, 40, 80, 40));

        trophy = new AnimatedTrophy();
        trophy.setAlignmentX(Component.CENTER_ALIGNMENT);
        add(trophy);
        add(Box.createVerticalStrut(40));

        victoryLabel = new JLabel("CHAMPION", SwingConstants.CENTER);
        victoryLabel.setFont(new Font("Segoe UI", Font.BOLD, 64));
        victoryLabel.setForeground(ACCENT_GOLD);
        victoryLabel.setAlignmentX(Component.CENTER_ALIGNMENT);
        add(victoryLabel);
        add(Box.createVerticalStrut(40));

        GlassPanel scoresPanel = new GlassPanel(new BorderLayout());
        scoresPanel.setMaximumSize(new Dimension(600, 300));
        scoresPanel.setBorder(BorderFactory.createTitledBorder(
            BorderFactory.createLineBorder(Color.WHITE, 1),
            "🏆 High Scores (Top 10)", TitledBorder.LEFT, TitledBorder.TOP,
            new Font("Segoe UI", Font.BOLD, 18), Color.WHITE));
        highScoresLabel = new JLabel("", SwingConstants.CENTER);
        highScoresLabel.setFont(new Font("Monospaced", Font.PLAIN, 16));
        highScoresLabel.setForeground(Color.WHITE);
    }
}

```

```
scoresPanel.add(highScoresLabel, BorderLayout.CENTER);
add(scoresPanel);
```

```
add(Box.createVerticalGlue());
```

```
HeroButton restartBtn = new HeroButton("▶ RESTART GAME", 300, 70);
restartBtn.setAlignmentX(Component.CENTER_ALIGNMENT);
restartBtn.addActionListener(e -> {
    trophy.stopVictoryAnimation();

    new StartPageDialog(parent).setVisible(true);
});
add(restartBtn);
}
```

```
public void updateWinner(String winnerName, List<GameEngine.ScoreEntry> highScores) {
    trophy.startVictoryAnimation();
    if (winnerName.contains("Draw")) {
        victoryLabel.setText("MATCH ENDED IN A DRAW");
        victoryLabel.setForeground(Color.GRAY);
    } else {
        victoryLabel.setText(winnerName.toUpperCase() + " WINS THE TOURNAMENT!");
        victoryLabel.setForeground(ACCENT_GOLD);
    }
    // Only show High Scores
    StringBuilder sb = new StringBuilder("<html><body style='text-align:center;'>");
    sb.append("<b>High Scores (Lowest Turns Win)</b><br>");
    for (int i = 0; i < highScores.size(); i++) {
        GameEngine.ScoreEntry entry = highScores.get(i);
        sb.append(String.format("<b>%d.</b> %s (%d turns)<br>", i + 1, entry.winnerName,
entry.turnsTaken));
    }
    sb.append("</body></html>");
    highScoresLabel.setText(sb.toString());
}
}
```

```
static class AnimatedTrophy extends JLabel {
    private javax.swing.Timer animationTimer;
```

```
private float rotation = 0;
```

```
public AnimatedTrophy() {  
    setText("🏆");  
    setFont(new Font("Segoe UI Emoji", Font.PLAIN, 180));  
    setHorizontalAlignment(SwingConstants.CENTER);  
    setPreferredSize(new Dimension(200, 200));  
    setMinimumSize(new Dimension(200, 200));  
}
```

```
public void startVictoryAnimation() {  
    if (animationTimer != null && animationTimer.isRunning()) {  
        animationTimer.stop();  
    }  
    animationTimer = new javax.swing.Timer(50, e -> {  
        rotation += 0.05f;  
        repaint();  
    });  
    animationTimer.start();  
}
```

```
public void stopVictoryAnimation() {  
    if (animationTimer != null) {  
        animationTimer.stop();  
        rotation = 0;  
        repaint();  
    }  
}
```

```
@Override
```

```
protected void paintComponent(Graphics g) {  
    Graphics2D g2d = (Graphics2D) g.create();  
    g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,  
RenderingHints.VALUE_ANTIALIAS_ON);
```

```
    int x = getWidth() / 2;  
    int y = getHeight() / 2;  
    g2d.rotate(rotation, x, y);  
    super.paintComponent(g2d);
```



```
        g2d.dispose();
    }
}
```

```
public static void main(String[] args) {
```

```
    try {
        UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
    } catch (Exception e) {

    }
}
```

```
SwingUtilities.invokeLater(() -> {
    new CricketAttaxEliteGUI();

    new StartPageDialog(null).setVisible(true);
});
}
```

```
public static class StartPageDialog extends JDialog {
    private final GameEngine engine = GameEngine.getInstance();
```

```
    public StartPageDialog(JFrame parent) {
        super(parent, "🏏 CRICKET ATTAX ELITE - Start Game", true);
        setLayout(new BorderLayout(20, 20));
        setSize(500, 450);
        setLocationRelativeTo(parent);
        setDefaultCloseOperation(DISPOSE_ON_CLOSE);
```

```
        JPanel contentPanel = new JPanel(new BorderLayout(20, 20));
        contentPanel.setBackground(new Color(20, 30, 40));
        contentPanel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
```

```
        JLabel titleLabel = new JLabel("🏏 CRICKET ATTAX ELITE 🏏", SwingConstants.CENTER);
        titleLabel.setFont(new Font("Arial", Font.BOLD, 28));
        titleLabel.setForeground(ACCENT_GOLD);
```

```
contentPanel.add(titleLabel, BorderLayout.NORTH);
```

```
JPanel mainPanel = new JPanel(new GridBagLayout());  
mainPanel.setOpaque(false);  
mainPanel.setBorder(BorderFactory.createEmptyBorder(40, 40, 40, 40));  
GridBagConstraints gbc = new GridBagConstraints();  
gbc.insets = new Insets(20, 15, 20, 15);  
gbc.anchor = GridBagConstraints.CENTER;
```

```
gbc.gridx = 0;  
gbc.gridy = 0;  
gbc.gridwidth = 1;  
JLabel nameLabel = new JLabel("Your Name:");  
nameLabel.setForeground(Color.WHITE);  
mainPanel.add(nameLabel, gbc);
```

```
gbc.gridx = 1;  
gbc.gridwidth = 2;  
gbc.fill = GridBagConstraints.HORIZONTAL;  
JTextField nameField = new JTextField(engine.getPlayerName(1), 20);  
nameField.setFont(new Font("Segoe UI", Font.PLAIN, 16));  
mainPanel.add(nameField, gbc);
```

```
gbc.gridx = 0;  
gbc.gridy = 1;  
gbc.gridwidth = 1;  
gbc.fill = GridBagConstraints.NONE;  
JLabel diffLabel = new JLabel("AI Difficulty:");  
diffLabel.setForeground(Color.WHITE);  
mainPanel.add(diffLabel, gbc);
```

```
gbc.gridx = 1;  
gbc.gridwidth = 2;  
gbc.fill = GridBagConstraints.HORIZONTAL;  
JComboBox<AIDifficulty> diffCombo = new JComboBox<>(AIDifficulty.values());  
diffCombo.setSelectedItem(engine.getAIDifficulty());  
diffCombo.setFont(new Font("Segoe UI", Font.PLAIN, 16));  
mainPanel.add(diffCombo, gbc);
```

```
contentPanel.add(mainPanel, BorderLayout.CENTER);
```

```
JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 20, 10));
```

```
buttonPanel.setOpaque(false);
```

```
JButton startButton = new JButton("🚀 START GAME", 200, 50);
```

```
startButton.addActionListener(e -> {
```

```
    String playerName = nameField.getText();
```

```
    AIDifficulty difficulty = (AIDifficulty) diffCombo.getSelectedItem();
```

```
    engine.setPlayer1Name(playerName);
```

```
    engine.setAIDifficulty(difficulty);
```

```
    if (parent instanceof CricketAttaxEliteGUI) {
```

```
        ((CricketAttaxEliteGUI) parent).startGame(playerName, difficulty);
```

```
    } else {
```

```
        engine.startGame();
```

```
    }
```

```
    dispose();
```

```
});
```

```
buttonPanel.add(startButton);
```

```
contentPanel.add(buttonPanel, BorderLayout.SOUTH);
```

```
this.setContentPane(contentPanel);
```

```
}
```

```
}
```

```
}
```

OUTPUT



Score: YKM 10 – 10 AI – HARD

YOUR TURN – Choose Wisely

KL Rahul (WICKETKEEPER)

Batting Avg	45.0
Strike Rate	138.0
Wickets Taken	0.0
Economy Rate	0.0
Fielding Catches	150.0
Experience Yrs	10.0

AI's Deck

Batting Avg	—.--
Strike Rate	—.--
Wickets Taken	—.--
Economy Rate	—.--
Fielding Catches	—.--
Experience Yrs	—.--

Batting Avg

Strike Rate

Wickets Taken

Economy Rate

Fielding Catches

Experience Yrs

Game Status / AI Rationale

YKM: Select a stat from your card.

End Mat...

Exit Ga...

Score: YKM 11 – 9 AI – HARD

YOUR TURN – Choose Wisely

Rashid Khan (ALLROUNDER)

Batting Avg	15.0
Strike Rate	140.0
Wickets Taken	200.0
Economy Rate	6.0
Fielding Catches	70.0
Experience Yrs	8.0

Steve Smith (BATSMAN)

Batting Avg	
Strike Rate	
Wickets Taken	
Economy Rate	
Fielding Catches	
Experience Yrs	

Batting Avg

Strike Rate

Wickets Taken

Economy Rate

Fielding Catches

Experience Yrs

Game Status / AI Rationale

AI – HARD Wins! 110.00 vs 70.00
AI Hard chose Wickets Taken (165.00) as it provides the largest comparative value on its card.

End Mat...

Exit Ga...

Score: YKM 8 – 12 AI – HARD

AI's TURN – Waiting for selection...

Pat Cummins (BOWLER)

Batting Avg	20.0
Strike Rate	110.0
Wickets Taken	190.0
Economy Rate	7.0
Fielding Catches	60.0
Experience Yrs	10.0

AI's Deck

Batting Avg	—.--
Strike Rate	—.--
Wickets Taken	—.--
Economy Rate	—.--
Fielding Catches	—.--
Experience Yrs	—.--

Batting Avg

Strike Rate

Wickets Taken

Economy Rate

Fielding Catches

Experience Yrs

Game Status / AI Rationale

PLAYER 2 AI is choosing stat...

End Mat...

Exit Ga...

AI – HARD WINS THE TOURNAMENT!

High Scores (Top 10)

High Scores (Lowest Turns Win)

1. Abrupt End – Draw (1 turns)

2. AI – HARD (11 turns)

3. AI – HARD (12 turns)

4. AI – HARD (37 turns)

5. AI – EASY (41 turns)

▶ RESTART GAME

Score: YKM 10 – 10 AI – HARD
YOUR TURN – Choose Wisely

Mitchell Starc (BOWLER)

Batting Avg12.0

Strike Rate105.0

Wickets Taken

Economy Rate

Fielding Catches

Experience Yrs

AI's Deck

Batting Avg--.--

Strike Rate--.--

Batting Avg

Strike Rate

Wickets Taken

Economy Rate

Fielding Catches

Experience Yrs

Game Status / AI Rationale

YKM: Select a stat from your card.

End Mat...

Exit Ga...

Confirm Exit (No Save)

Exit now? This will quit immediately and WILL NOT save or alter the current match. Continue?

No

Yes

6. References

1. Oracle. (2024). *The Java™ Tutorials*. Oracle Documentation.
<https://docs.oracle.com/javase/tutorial/>
2. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
3. Eckstein, R., Loy, M., & Wood, D. (1998). *Java Swing*. O'Reilly Media
4. GeeksforGeeks. (2023). *Introduction to Java Swing*.
<https://www.geeksforgeeks.org/java-swing/>
5. GeeksforGeeks. (2023). *Java Swing | Creating a GUI Application*.
<https://www.geeksforgeeks.org/java-swing-simple-user-registration-form/>